

Section 1- Design

Q1. What do you consider a 'state' in this environment? How would you define a 'neighboring state' for this problem?

A state is the current grid configuration (the $n \times n$ array where each cell is -1 for empty or 0-3 for a color) along with the list of placed shapes, where each shape records its type, position, and color.

A neighboring state is any state reachable by a single logical move from the current state. This includes undoing a placed shape and replacing it with a different color, undoing a placed shape and replacing it with a different shape type, or undoing multiple 1x1 shapes and replacing them with a single larger shape that covers the same cells.

Q2. First-choice local search requires that you move to a random neighboring state. How do you (plan to) implement random neighbor selection? Explain in 1-2 sentences.

To generate a random neighbor, I randomly pick a large shape type 1-8 and a random valid grid position. I check if all cells the shape would cover are the same color and are individually placed as 1x1 shapes. If so, I undo those 1x1 shapes and place the single large shape instead, reducing the total shape count.

Q3. Specify your objective function. Are all constraints modeled through the objective function, or are some treated as explicit rules? Explain your choices.

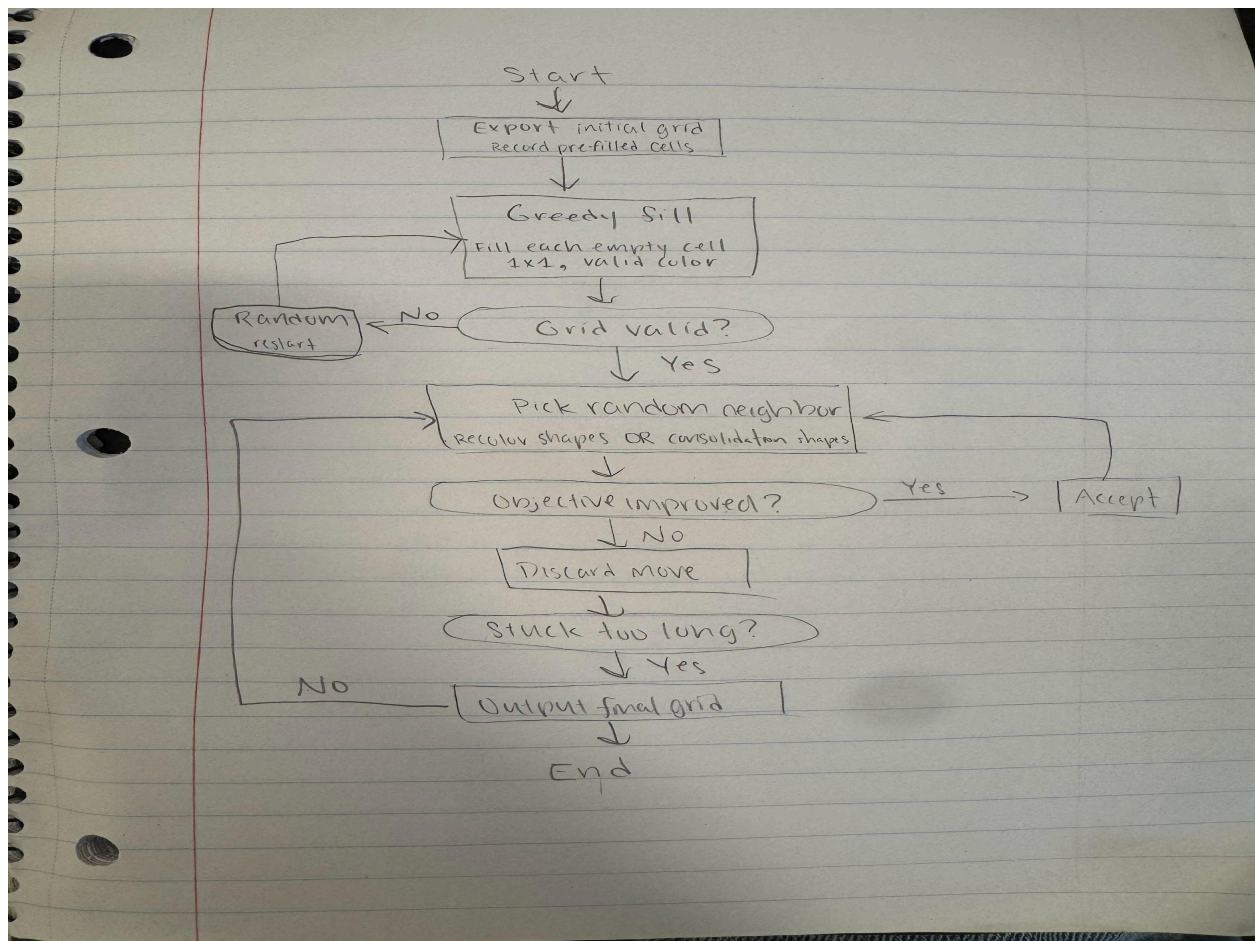
My objective function is

$f(\text{state}) = \text{number of shapes placed}$

The goal is to minimize this value. Since the algorithm starts with a valid coloring from the greedy fill and only accepts moves that maintain validity, the objective only needs to track shape count.

Both constraints are treated as explicit rules. The "all cells must be filled" constraint is enforced by completing the greedy fill before hill climbing begins. The "no adjacent same colors" constraint is enforced by only accepting neighbor moves that don't introduce violations. I use `getAvailableColor()` when recoloring and verify validity before accepting any consolidation moves.

Q4. Draw an end-to-end overview diagram/flowchart for your (proposed) local search approach. If using any extensions to local search (such as random bad moves or restarts), that workflow must be reflected in your flowchart. [May be hand-drawn and scanned into the PDF.]



Section 3- Analysis/Reflections

Q1. Did you encounter local optima when testing your implementation? If so, when did this happen (structurally, what was going on with the state)? If not, provide an example of a local optimum that could get the agent stuck, where the only way out would be to backtrack to a previous state.

Q2. Is your solution optimal or near-optimal? Explain why/why not